

Естественно, в игре должно быть какое-никакое звуковое сопровождение. Звук добавляет ощущение взаимодействия - звуки наведения на кнопки, нажатия, покупка юнитов, улучшение башни. Фоновая музыка может задавать настроение.

SoundSO.

Создаём конфиг ScriptableObject для звука. Он будет хранить название, сам звуковой клип, громкость клипа (должна быть относительно выровнена), зацикленность и тип звука.

SoundType представляет все типы звуков - Music, SFX, LoopSFX, UI. Позже каждый из них будет обрабатываться по разному.

SoundEnum был сделан для того, чтобы были универсальные звуки, дабы не прокидывать во все кнопки один и тот же SoundSO. При наведении будем просто вызывать PlaySFX(SoundEnum).

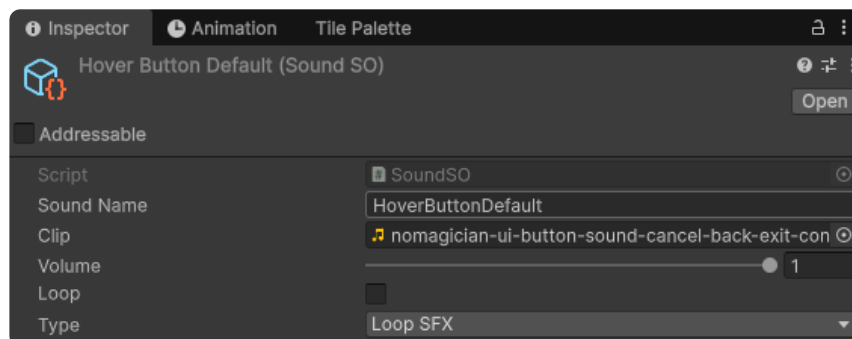
MusicType - тип принадлежности фоновой музыки. Для боевых сцен может быть массив разных аудио. При запуске уровня музыка будет выбираться случайно из доступных.

```
1 using UnityEngine;
2
3 [CreateAssetMenu(menuName = "Audio/Sound")]
4 public class SoundSO : ScriptableObject
5 {
6     public string soundName;
7     public AudioClip clip;
8     [Range(0, 1)] public float volume = 1f;
9     public bool loop = false;
10    public SoundType type = SoundType.SFX;
11
12    public bool limited = false;
13    public int maxSimultaneous = 5;
14 }
15
16 public enum SoundType
17 {
18     Music,
19     SFX,
20     LoopSFX,
21     UI
22 }
23
24 public enum SoundDefaultEnum
25 {
26     ButtonClick,
27     ButtonHover,
```

```

28     ButtonError,
29
30     UnitBuy,
31     UnitUpgrade,
32     UnitMaxUpgrade,
33
34     TowerBuy,
35     TowerUpgrade,
36     TowerUpgradeMax,
37     TowerCrash,
38
39     EnemyAttack,
40     EnemyDie1,
41     EnemyDie2,
42     EnemyDie3,
43     EnemyDie4,
44
45     Taymer,
46
47     WaveStart,
48     WinFloor,
49     GameOver
50 }
51
52 public enum MusicType
53 {
54     Menu,
55     FightRandom,
56     Boss
57 }

```



SoundSO.

AudioSourcePool.

Учитывая, что работаем мы с SO. Нам всё таки придётся создавать AudioSource типы и записывать в них конфиг из SO. Чтобы избежать множества аллокаций, частых сборок мусора, мы сделаем отдельный пул для аудио сурс. Логика этого пула отличается от обычного GameObject, который мы использовали для врагов, поэтому он будет создан отдельно.

```

1 using System.Collections.Generic;
2 using UnityEngine;
3
4 public class AudioSourcePool : MonoBehaviour

```

```

5 {
6     private Queue pool = new Queue();
7     private Transform parent;
8     private int expandSize;
9     private GameObject sourcePrefab;
10
11     private HashSet activeSources = new HashSet
12     public IReadOnlyCollection ActiveSources => activeSour
13
14     public AudioSourcePool(GameObject prefab, int initialSize, int expa
15     {
16         this.parent = parent;
17         this.expandSize = Mathf.Max(10, expandSize);
18         sourcePrefab = prefab;
19         CreateBatch(initialSize);
20     }
21
22     private void CreateBatch(int count)
23     {
24         for (int i = 0; i < count; i++)
25         {
26             GameObject go = GameObject.Instantiate(sourcePrefab, parent
27             go.SetActive(false);
28             pool.Enqueue(go.GetComponent());
29         }
30     }
31
32     public AudioSource Get()
33     {
34         if (pool.Count == 0) CreateBatch(expandSize);
35
36         AudioSource source = pool.Dequeue();
37         source.gameObject.SetActive(true);
38
39         activeSources.Add(source);
40
41         return source;
42     }
43
44     public void Return(AudioSource source)
45     {
46         if (!activeSources.Remove(source))
47             return;
48
49         source.Stop();
50         source.clip = null;
51         source.pitch = 1f;
52
53         source.gameObject.SetActive(false);
54
55         pool.Enqueue(source);
56     }
57 }

```

AudioManager.

```

1 using NUnit.Framework;
2 using System.Collections;
3 using System.Collections.Generic;
4 using System.Linq;
5 using Unity.VisualScripting;
6 using UnityEditor.AddressableAssets.Build;
7 using UnityEngine;
8
9

```

```

10 public class AudioManager : Manager<AudioManager>
11 {
12     [Header("AudioSource Prefab")]
13     public GameObject audioSourcePrefab;
14
15     [Header("Pool Settings")]
16     public int initialPoolSize = 10;
17     public int expandPoolSize = 10;
18
19     private AudioSourcePool sfxPool;
20     private readonly Dictionary<SoundSO, AudioSource> loopSources = ne
21
22     private Dictionary<SoundSO, int> playingSFX = new();
23
24     [Header("Music Sources for crossfade")]
25     private AudioSource musicSourceA;
26     private AudioSource musicSourceB;
27     private bool usingSourceA = true;
28
29     private float musicVolume = 1f;
30     private float sfxVolume = 1f;
31     private Coroutine musicCoroutine;
32     private float currentPitch = 1f;
33
34     [Header("UISource")]
35     private AudioSource uiSource;
36
37     [Header("AudioLibrary")]
38     [SerializeField] private AudioLibrary library;
39
40
41
42
43     protected override void Awake()
44     {
45         base.Awake();
46
47         sfxPool = new AudioSourcePool(audioSourcePrefab, initialPoolSi
48
49         musicSourceA = new GameObject("MusicSourceA").AddComponent<Aud
50         musicSourceB = new GameObject("MusicSourceB").AddComponent<Aud
51         musicSourceA.transform.SetParent(transform);
52         musicSourceB.transform.SetParent(transform);
53
54         musicSourceA.loop = true;
55         musicSourceB.loop = true;
56
57         uiSource = new GameObject("UISource").AddComponent<AudioSource
58         uiSource.transform.SetParent(transform);
59         uiSource.loop = false;
60         uiSource.ignoreListenerPause = true;
61
62         library.Init();
63     }
64
65     private void Start()
66     {
67         LoadSettings();
68
69         SettingsService.Instance.OnMusicVolumeChanged += SetMusicVolum
70         SettingsService.Instance.OnSFXVolumeChanged += SetSFXVolume;
71
72         SpeedGameManager.Instance.OnSpeedGameChanged += HandleSpeedCha
73     }
74
75     private void LoadSettings()
76     {

```

```

77         musicVolume = SettingsService.Instance.MusicVolume;
78         sfxVolume = SettingsService.Instance.SFXVolume;
79
80         musicSourceA.volume = musicVolume;
81         musicSourceB.volume = musicVolume;
82     }
83
84     //////////////////////////////////////////////////
85     ///      SFX
86     //////////////////////////////////////////////////
87
88
89     // PlaySFX(SoundS0) - одновременно производит звук типа SFX, а также
90     // корутину, которая по завершению звука вернёт компонент в пул.
91
92     public void PlaySFX(SoundS0 sound)
93     {
94         if (sound.type != SoundType.SFX)
95             return;
96
97         bool useLimit = sound.limited && sound.maxSimultaneous > 0;
98
99         if (useLimit)
100         {
101             if (playingSFX.TryGetValue(sound, out int count))
102             {
103                 if (count >= sound.maxSimultaneous)
104                     return;
105             }
106             else
107             {
108                 playingSFX[sound] = 0;
109             }
110         }
111
112         AudioSource source = sfxPool.Get();
113         source.clip = sound.clip;
114         source.volume = sound.volume * sfxVolume;
115         source.loop = false;
116         source.pitch = currentPitch;
117         source.Play();
118
119         if (useLimit)
120         {
121             playingSFX[sound]++;
122             StartCoroutine(ReturnLimitedSFXAfterPlay(source, sound));
123         }
124         else
125         {
126             StartCoroutine(ReturnToPoolAfterPlay(source));
127         }
128     }
129
130     private IEnumerator ReturnToPoolAfterPlay(AudioSource source)
131     {
132         yield return new WaitWhile(() => source.isPlaying);
133         sfxPool.Return(source);
134     }
135
136     //////////////////////////////////////////////////
137     ///      SFXType
138     //////////////////////////////////////////////////
139
140     public void PlaySFXType(SoundDefaultEnum sound)
141     {
142         var clip = library.GetClip(sound);
143         if (clip != null)

```

```

144     {
145         PlaySFX(clip);
146     }
147 }
148 /*
149  PlaySFXType(SoundEnum) - запускает звук из библиотеки частых звуков.
150 Например, для кнопок у меня есть два стандартных звука. И мне не придётся
151 указывать на них ссылку в скрипте кнопок, а достаточно будет вызвать э
152 и передать соответствующий тип.
153 */
154
155     ////////////////
156     ///    LimitedSFX
157     ////////////////
158     ///
159
160     private IEnumerator ReturnLimitedSFXAfterPlay(AudioSource source,
161     {
162         yield return new WaitWhile(() => source.isPlaying);
163
164         if (playingSFX.TryGetValue(sound, out int count))
165         {
166             count--;
167
168             if (count <= 0)
169                 playingSFX.Remove(sound);
170             else
171                 playingSFX[sound] = count;
172         }
173         sfxPool.Return(source);
174     }
175
176     ////////////////
177     ///    UISFX
178     ////////////////
179
180     // PlayUISFX(SoundS0) - одновременно производит звук UI типа.
181     // Отличие его в том, что он играет одинаково вне зависимости от скоро
182     // Более того, пауза не прерывает его проигрывание.
183
184     public void PlayUISFX(SoundS0 sound)
185     {
186         if (sound.type != SoundType.UI) return;
187
188         uiSource.pitch = 1f;
189         uiSource.PlayOneShot(sound.clip, sound.volume * sfxVolume);
190     }
191
192     public void PlayUISFXType(SoundDefaultEnum sound)
193     {
194         var clip = library.GetClip(sound);
195         if (clip != null)
196         {
197             PlayUISFX(clip);
198         }
199     }
200
201     // PlayUISFXType(SoundEnum) - проигрывает именно UISFX (как-раз кнопки
202     // Позволяет производить звуки даже на паузе.
203
204     ////////////////
205     ///    LoopSFX
206     ////////////////
207
208
209     /*
210     PlayLoopSFX(SoundS0) - начинает произведение зацикленного звука

```

```

211 (лазерная башня, к примеру). Их мы храним в отдельной коллекции,
212 чтобы можно было в случае чего поставить на паузу и возобновить
213 (когда игрок нажмёт паузу).
214 */
215     public AudioSource PlayLoopSFX(SoundS0 sound)
216     {
217         if (sound.type != SoundType.LoopSFX)
218             return null;
219
220         AudioSource source = sfxPool.Get();
221
222         source.clip = sound.clip;
223         source.volume = sound.volume * sfxVolume;
224         source.loop = true;
225         source.pitch = currentPitch;
226
227         source.Play();
228
229         loopSources[sound] = source;
230
231         return source;
232     }
233
234     public void StopLoopSFX(SoundS0 sound)
235     {
236         if (!loopSources.TryGetValue(sound, out var source))
237         {
238             return;
239         }
240
241         source.Stop();
242         sfxPool.Return(source);
243
244         loopSources.Remove(sound);
245     }
246
247     public void PauseLoopSFX(SoundS0 sound)
248     {
249         if (loopSources.TryGetValue(sound, out var source))
250         {
251             source.Pause();
252         }
253     }
254     public void ResumeLoopSFX(SoundS0 sound)
255     {
256         if (loopSources.TryGetValue(sound, out var source))
257             source.UnPause();
258     }
259
260     private void PauseLoopSFX()
261     {
262         foreach (var source in loopSources.Values)
263             source.Pause();
264     }
265
266     private void ResumeLoopSFX()
267     {
268         foreach (var source in loopSources.Values)
269             source.UnPause();
270     }
271
272     private void StopAllSFX()
273     {
274         var active = new List<AudioSource>(sfxPool.ActiveSources);
275
276         foreach (var source in active)
277             {

```

```

278         if (loopSources.ContainsValue(source))
279             continue;
280
281         source.Stop();
282         sfxPool.Return(source);
283     }
284 }
285
286 // StopAllSFX() - берёт все активные звуки. Сверяет их с массивом записей
287 // и если они совпадают, то пропускает, т.к. для них у нас своя логика
288
289 ///////////////////////////////////////////////////
290 ///     LoopSFXType
291 ///////////////////////////////////////////////////
292
293
294 public void PlayLoopSFXType(SoundDefaultEnum soundType)
295 {
296     var clip = library.GetClip(soundType);
297
298     if (clip != null)
299         PlayLoopSFX(clip);
300 }
301
302 public void StopLoopSFXType(SoundDefaultEnum soundType)
303 {
304     var clip = library.GetClip(soundType);
305
306     if (clip != null)
307         StopLoopSFX(clip);
308 }
309
310 public void PauseLoopSFXType(SoundDefaultEnum soundType)
311 {
312     var sound = library.GetClip(soundType);
313
314     if (sound != null)
315         PauseLoopSFX(sound);
316 }
317
318 public void ResumeLoopSFXType(SoundDefaultEnum soundType)
319 {
320     var sound = library.GetClip(soundType);
321
322     if (sound != null)
323         ResumeLoopSFX(sound);
324 }
325
326 ///////////////////////////////////////////////////
327 ///     Music
328 ///////////////////////////////////////////////////
329
330 /*
331  PlayMusic(SoundS0, float) - запускает проигрывание музыки со звуковым
332  Мы берём текущую активную музыку, а также ту, что нам предстоит включить
333  Устанавливаем громкость второй на 0 и начинаем проигрывание.
334  Сразу прокидываем их в корутину CrossFadeMusic.
335  */
336 public void PlayMusic(SoundS0 music, float fadeDuration = 2f)
337 {
338     if (music.type != SoundType.Music) return;
339
340     AudioSource active = usingSourceA ? musicSourceA : musicSource;
341     AudioSource next = usingSourceA ? musicSourceB : musicSourceA;
342
343     next.clip = music.clip;
344     next.volume = 0;

```

```

345     next.loop = music.loop;
346     next.Play();
347
348     musicCoroutine = StartCoroutine(CrossFadeMusic(active, next, f
349     usingSourceA = !usingSourceA;
350 }
351
352 /*
353 CrossFadeMusic(AudioSource, AudioSource, float) - за период перехода
354 относительно deltaTime, через Lerp плавно сбрасывает громкость одной,
355 и прибавляет громкость другой. После чего останавливает первую.
356 */
357
358     private IEnumerator CrossFadeMusic(AudioSource from, AudioSource to
359     {
360         float t = 0f;
361         while (t < duration)
362         {
363             t += Time.deltaTime;
364             from.volume = Mathf.Lerp(musicVolume, 0, t / duration);
365             to.volume = Mathf.Lerp(0, musicVolume, t / duration);
366             yield return null;
367         }
368         from.Stop();
369         from.clip = null;
370         to.volume = musicVolume;
371
372         musicCoroutine = null;
373     }
374
375     public void PlayMusicType(MusicType type)
376     {
377         var music = library.GetMusic(type);
378         if (music != null)
379         {
380             PlayMusic(music);
381         }
382     }
383
384     // PlayMusicType(MusicType) - можно также прокинуть случайную из досту
385     // фоновую музыку по её типу.
386
387     public void PauseMusic()
388     {
389         if (musicSourceA.isPlaying) musicSourceA.Pause();
390         if (musicSourceB.isPlaying) musicSourceB.Pause();
391     }
392
393     public void ResumeMusic()
394     {
395         musicSourceA.UnPause();
396         musicSourceB.UnPause();
397     }
398
399     // PauseMusic() - ставит играющий звук на паузу.
400     // ResumeMusic() - UnPause.
401
402     // StopMusic(float) - плавно останавливает музыкальное сопровождение.
403
404     public void StopMusic(float fadeDuration = 1.5f)
405     {
406         if (musicCoroutine != null)
407         {
408             StopCoroutine(musicCoroutine);
409             musicCoroutine = null;
410         }
411

```

```

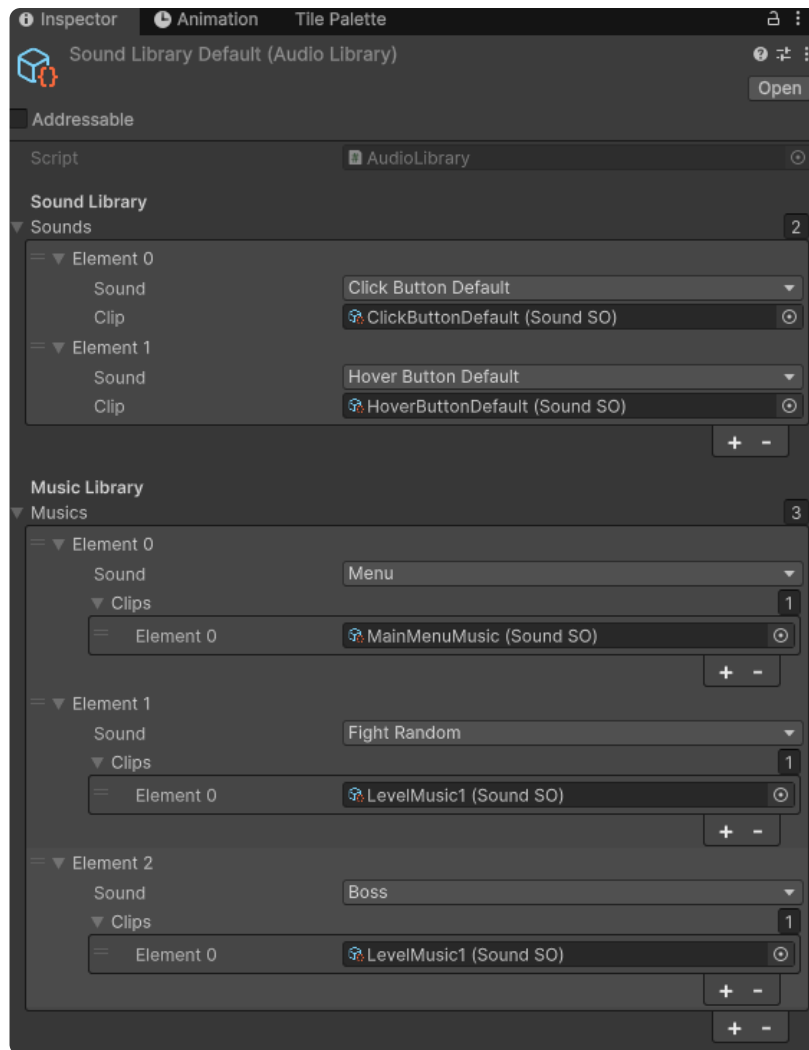
412         if (musicSourceA.isPlaying)
413             StartCoroutine(FadeOutAndStop(musicSourceA, fadeDuration))
414
415         if (musicSourceB.isPlaying)
416             StartCoroutine(FadeOutAndStop(musicSourceB, fadeDuration))
417     }
418
419     // FadeOutAndStop(AudioSource, float) - как раз таки берёт текущее соп
420     // и плавно скручивает его громкость, а позже останавливает.
421
422     private IEnumerator FadeOutAndStop(AudioSource source, float fadeD
423     {
424         float startVolume = source.volume;
425         float t = 0f;
426
427         while (t < fadeDuration)
428         {
429             t += Time.deltaTime;
430             source.volume = Mathf.Lerp(startVolume, 0, t / fadeDuratio
431             yield return null;
432         }
433
434         source.Stop();
435         source.clip = null;
436         source.volume = musicVolume;
437     }
438
439     ////////////////////////////////////
440     ///      SetVolume
441     ////////////////////////////////////
442     ///
443
444     public void SetMusicVolume(float value)
445     {
446         musicVolume = value;
447         musicSourceA.volume = usingSourceA ? musicVolume : 0;
448         musicSourceB.volume = usingSourceA ? 0 : musicVolume;
449     }
450
451     public void SetSFXVolume(float value) => sfxVolume = value;
452
453     /*
454     SetMusicVolume(float) - устанавливает громкость текущей активной муз
455     SetSFXVolume - устанавливает громкость для SFX.
456     */
457
458
459     private void HandleSpeedChanged(GameSpeed speed)
460     {
461         if (speed == GameSpeed.Pause)
462         {
463             PauseAllGameAudio();
464             return;
465         }
466
467         ResumeMusic();
468         ResumeLoopSFX();
469         SetPitchBySpeed(SpeedGameManager.Instance.SpeedMultiplier);
470     }
471
472     /*
473     Мы подписались на изменение скорости игры. Событие, которое вызывает
474     SpeedGameManager. В этом методе мы вызываем методы ResumeMusic, Resume
475     SetPitchBySpeed.
476     */
477
478     private void PauseAllGameAudio()

```

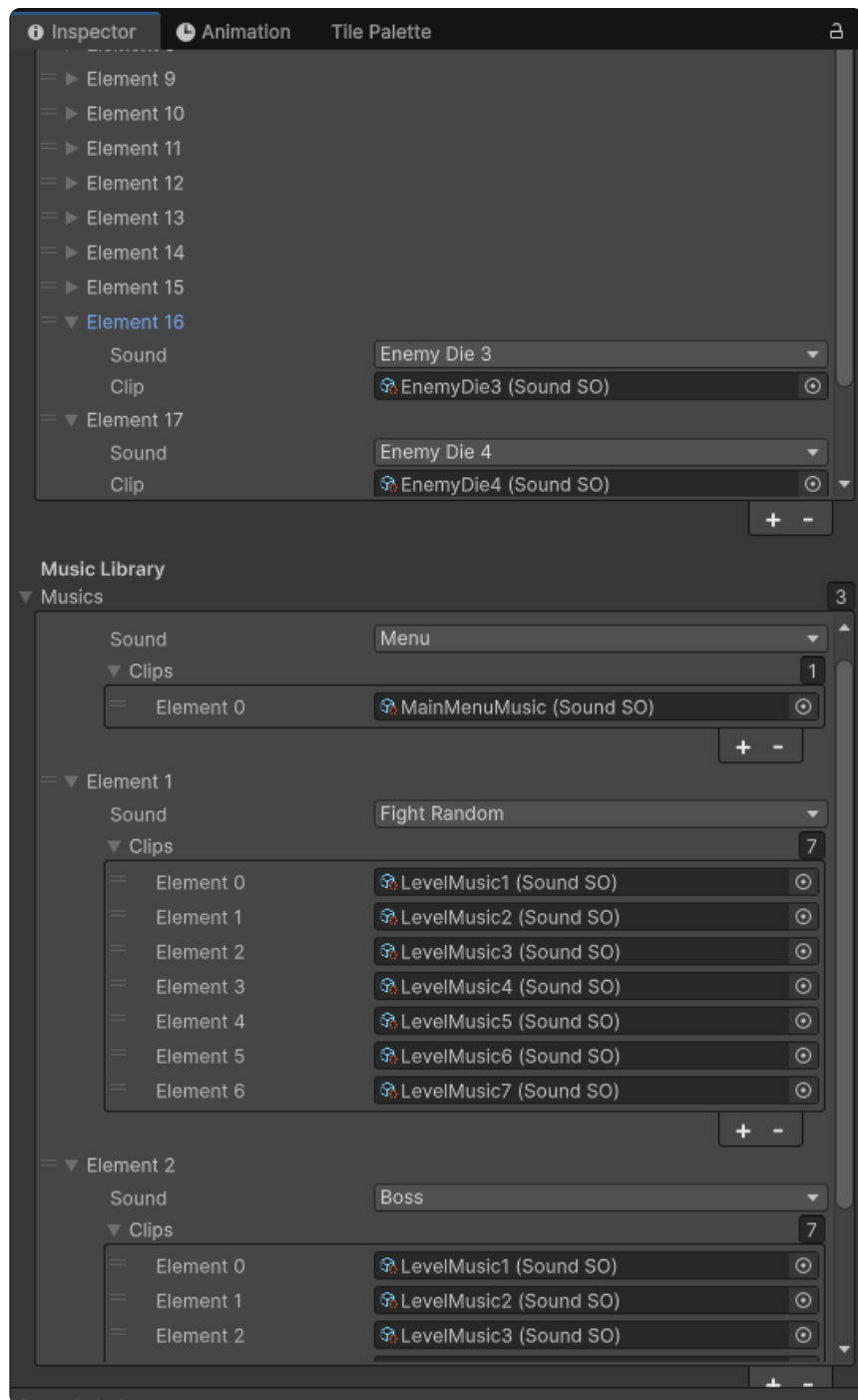
```

479     {
480         PauseMusic();
481         StopAllSFX();
482         PauseLoopSFX();
483     }
484
485     // PauseAllGameAudio() - ставит на паузу все существующие звуки.
486     // При этом короткие звуки не ставятся на паузу, а выключаются.
487
488
489
490     private void SetPitchBySpeed(float speed)
491     {
492         if (speed <= 1f)
493             currentPitch = 1f;
494         else
495             currentPitch = Mathf.Lerp(1f, 1.5f, (speed - 1f) / 3); //
496
497         foreach (var source in sfxPool.ActiveSources)
498         {
499             source.pitch = currentPitch;
500         }
501     }
502 }
503
504 /*
505     SetPitchBySpeed(float) - ускоряет скорость аудио, но не соразмерно с
506     Если ставить на x4 скорость и делать 4.0 Pitch, то звук будет вообще н
507     Поэтому через Lerp мы вычисляем немного иное значение. На x4 скорости
508     будет равен 1.5.
509     */

```



Библиотека стандартных звуков и библиотека музыкального сопровождения.



Обновленное хранилище звуков.

Сейчас вызов звуков работает через жесткую связность. Вызывать приходится звук напрямую через `AudioManager.Instance.PlaySFX()`. Для небольшого проекта это нормально. В более крупных проектах используют Ievent-Driven систему (IeventBus).

Если враг умирает, то он прокидывает звук в глобальный менеджер. Враг знает про AudioManager. В IeventDriven враг при смерти вызывает событие, куда передаёт данные.

Допустим у нас есть статический класс AudioEvents с событием PlaySFX(AudioSO). В таком случае AudioManager подписывается на событие и будет проигрывать любой пришедший звук, не зная даже от кого он пришёл. А враг, при смерти будет вызывать AudioEvents.PlaySFX(stats.deathSound); И враг ничего не знает о менеджере. У них есть прослойка. Враг не знает методов и способов воспроизведения звука. А менеджер может меняться и просто реагировать на абстрактное событие. К тому же, какой-нибудь отладочный класс для логгирования также сможет просто подписаться на событие.

- ✓ Аудио менеджер ещё может поменяться, но даже сейчас он отлично функционирует и его достаточно.